Semi Senior

As a Semi Senior Data Scientist you should be able to expand on the above by knowing about the following topics:

Classical Algorithms

- Linear regression
  - Logistic Regression

  Decision Tree

  Random Forest k-Means

  KNN

  Shallow NN

  SVM
  -

Statistics / Math

- Central Limit Theorem (Can you explain it in simple terms? Why is it useful?)

Cross validation (Explain the technique., Why do we need it? what is data leakage? why partition data?)

Metrics for classification problems: confusion matrix, accuracy, precision, recall, F1 score.

Metrics for regression problems: MAE, MAPE, RMSE.

- Gradient descent (How does it work? Soft requirement on convexity? When does it not matter?)

  What is a convolution?

  Hypothesis generation.

  Error analysis.

  Explain how the autocorrelation of a positive-trend line can be negative.

  ANOVA, multivariate analysis, PCA / Factor Analysis, population tests and comparison.

Machine Learning

- Algorithm customization
  - What do you do if you are trying to do regression and you have heteroskedasticity?
  - How do you avoid overfitting?
  - How do you add non-linearities to a regression?

  How do you add classes to a regression?

Variable selection / feature engineering (What is it? How would you do it?)

- Handling "too many features" (Is that a problem? When? What would you do about it?)

How can you choose the value of k on k-means?

Transformer Basics: Understands the "Encoder-only" (BERT) vs. "Decoder-only" (GPT)

Understands Embeddings as an extension to "Feature Engineering."
LLM
Workflow vs. Agent: Understands the difference between a fixed Workflow (Step A -> Step
B) and a dynamic Agent (Goal -> Reason -> Action).
-Vector DB & other types of Retrieval: identifying when to use each based on trade-offs in latency, scale, and metadata filtering requirements.
Encoding & Token Awareness: Understands how different models "count" tokens and how that impacts both cost and performance.
- Context Precision & Recall: Did we retrieve only relevant documents? Did the retrieval step actually include the document containing the answer?
Add context to the model using prompting techniques: Zero and few shots, prompt chaining, etc.
Software Development

- What is the difference between "checkout", "add", "commit" and "branch" in Git?
  - What are the 4 main parts of an SQL statement?

Python

- Basic API development: FastAPI, Flask, Eventlet / Gunicorn. REST, wsgi.
  - Visualizations: Bokeh, Plotly, others.
    Fundamentals of Object Oriented Programming.
    Packaging classes and methods into data science libraries.
    Literate programming.
    Reproducible research.
    Error catching, context manager, generators and comprehensions, reusable methods and classes, some code optimization and profiling.
    Basic Python Engineering: Able to write clean functions that can be used as "Tools" by an AI.
  - Basic Agent Tool: LangChain, Smolagents, etc.

https://scholar.google.com/citations?user=6dskOSUAAAAJ&hl=en : https://ai-plans.com/file_storage/4f32fa39-3a01-46c7-878e-c92b7aa7165f_2212.08073v1.pdf

LLM and Generative AI
-Knows and use of generative AI-based tools.
Basic prompting techniques. Prompt Hygiene.

Classical Algorithms

- Linear regression

1. Linear Regression

Model

$$\hat{y} = w^T x + b$$

Optimization objective (least squares)

$$\min_{w,b} \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Variables

Symbol	Meaning
x_i	input feature vector
y_i	true value
w	weight vector
b	bias
n	number of samples

Training algorithm (Gradient Descent)

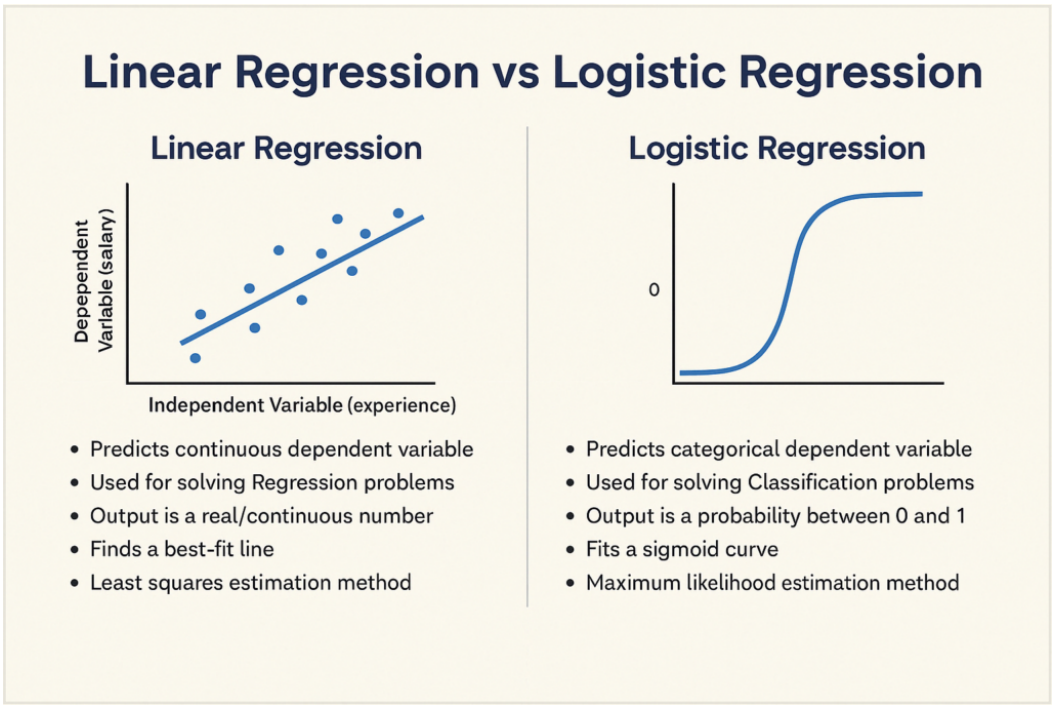
1. Initialize w, b
2. Compute predictions
   $\hat{y}_i = w^T x_i + b$
3. Compute loss (MSE)
4. Update parameters

$$w \leftarrow w - \eta \nabla_w L$$

$$b \leftarrow b - \eta \nabla_b L$$

5. Repeat until convergence

Symbol	Meaning
η	learning rate



Goal

Binary classification (predict probability of a class).

Model

$$P(y = 1|x) = \sigma(w^T x + b)$$

Sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Variables

Symbol	Meaning
x	input features
w	weights
b	bias
(P(y=1	x))

3. Decision Tree

Goal

Split data into regions using **feature thresholds**.

Split criterion (example: Gini)

$$G = 1 - \sum_{k=1}^{K} p_k^2$$

Variables

Symbol	Meaning
K	number of classes
p_k	probability of class k in node

Lower Gini → better split.

Gini impurity [edit]

Gini impurity, **Gini's diversity index**,[26] or **Gini-Simpson Index** in biodiversity research, is used by the CART (classification and regression tree) algorithm for classification trees. Gini impurity is the probability that a randomly chosen element of a set would be mislabeled if it were labeled randomly and independently according to the distribution of labels in the set. It reaches its minimum (zero) when all cases in the node fall into a single target category.

For a set of items with J classes and relative frequencies p_i, $i \in \{1, 2, \ldots, J\}$, the probability of choosing an item with label i is p_i, and the probability of miscategorizing that item is $\sum_{k \neq i} p_k = 1 - p_i$. The Gini impurity is computed by summing pairwise products of these probabilities for each class label:

$$\mathrm{I}_G(p) = \sum_{i=1}^{J} \left(p_i \sum_{k \neq i} p_k \right) = \sum_{i=1}^{J} p_i(1 - p_i) = \sum_{i=1}^{J} (p_i - p_i^2) = \sum_{i=1}^{J} p_i - \sum_{i=1}^{J} p_i^2 = 1 - \sum_{i=1}^{J} p_i^2.$$

The Gini impurity is also an information theoretic measure and corresponds to Tsallis Entropy with deformation coefficient $q = 2$, which in physics is associated with the lack of information in out-of-equilibrium, non-extensive, dissipative and quantum systems. For the limit $q \to 1$ one recovers the usual Boltzmann-Gibbs or Shannon entropy. In this sense, the Gini impurity is nothing but a variation of the usual entropy measure for decision trees.

4. Random Forest

Goal

Ensemble of many decision trees trained on random subsets.

Prediction

$$\hat{y} = \frac{1}{T} \sum_{t=1}^{T} h_t(x)$$

Variables

Symbol	Meaning
T	number of trees
$h_t(x)$	prediction of tree t
$\hat{y}$	final prediction

Idea: **reduce variance by averaging trees**.

5. k-Means

Goal

Cluster data into **k groups**.

Objective

$$\min \sum_{i=1}^{n} ||x_i - \mu_{c_i}||^2$$

Variables

Symbol	Meaning
x_i	data point
μ_{c_i}	centroid of assigned cluster
c_i	cluster label
k	number of clusters

Algorithm steps:

1. Initialize centroids
2. Assign points to nearest centroid
3. Update centroids
4. Repeat

6. k-Nearest Neighbors (KNN)

Goal

Classify based on **closest neighbors**.

Distance (Euclidean)

$$d(x, x_i) = \sqrt{\sum_{j=1}^{m} (x_j - x_{ij})^2}$$

Variables

Symbol	Meaning
x	query point
x_i	training point
m	number of features
k	number of neighbors

Prediction = **majority vote of k nearest samples**.

7. Shallow Neural Network

Goal

Learn nonlinear mapping with **one hidden layer**.

Model

Hidden layer:

$$h = \sigma(W_1 x + b_1)$$

Output layer:

$$y = W_2 h + b_2$$

Variables

Symbol	Meaning
x	input vector
W_1, W_2	weight matrices
b_1, b_2	biases
h	hidden layer activation
σ	activation function

8. Support Vector Machine (SVM)

Goal

Find the **hyperplane that maximizes margin between classes**.

Decision boundary

$$w^T x + b = 0$$

Optimization

$$\min_{w,b} \frac{1}{2}||w||^2$$

subject to

$$y_i(w^T x_i + b) \geq 1$$

Variables

Symbol	Meaning
x_i	data point
y_i	class label (-1, +1)
w	normal vector of hyperplane
b	bias

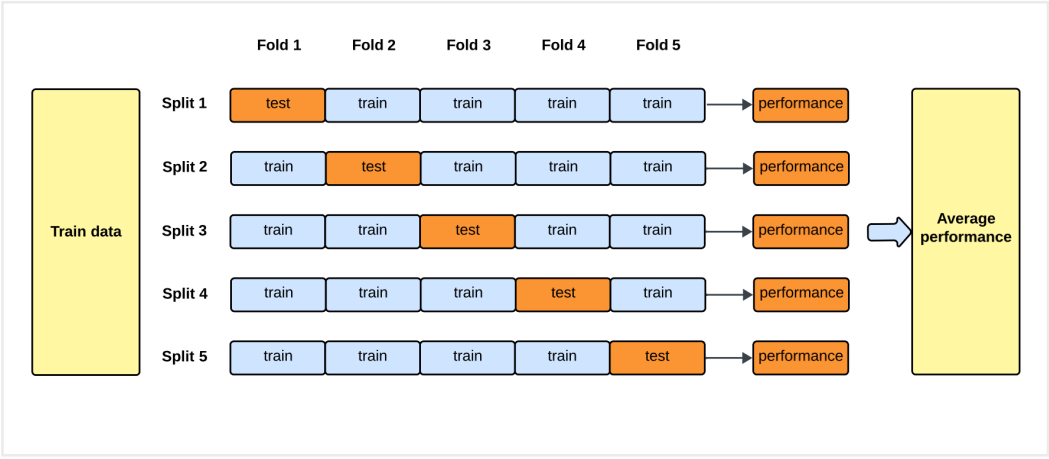
Algorithm	Type	Key Idea
Linear Regression	Regression	Fit linear function
Logistic Regression	Classification	Sigmoid probability
Decision Tree	Classification/Regression	Recursive splits
Random Forest	Ensemble	Average many trees
k-Means	Clustering	Minimize distance to centroids
KNN	Classification	Vote of nearest neighbors
Shallow NN	Nonlinear model	Hidden layer transformations
SVM	Classification	Maximum margin hyperplane

Central Limit Theorem (Can you explain it in simple terms? Why is it useful?

In probability theory, the **central limit theorem** (**CLT**) states that, under appropriate conditions, the distribution of a normalized version of the sample mean converges to a standard normal distribution. This holds even if the original variables themselves are not normally distributed. There are several versions of the CLT, each applying in the context of different conditions.

In probability theory, the **law of large numbers** is a mathematical law that states that the average of the results obtained from a large number of independent random samples converges to the true value, if it exists.

Cross validation (Explain the technique., Why do we need it? what is data leakage? why partition data?)



Data leakage in machine learning occurs when a model uses information during

training that wouldn't be available at the time of prediction. Leakage causes a predictive model to look accurate until deployed in its use case; then, it will yield inaccurate results, leading to poor decision-making and false insights.

Metrics for classification problems: confusion matrix, accuracy, precision, recall, F1 score.

Key Evaluation Metrics Defined

- **Accuracy** ($\frac{TP+TN}{TP+TN+FP+FN}$)**:** The ratio of correctly predicted observations to total observations. It works best with balanced data.
- **Precision** ($\frac{TP}{TP+FP}$)**:** The ratio of correctly predicted positive observations to total predicted positives. High precision is essential when false positives are costly, such as in spam detection.
- **Recall (Sensitivity)** ($\frac{TP}{TP+FN}$)**:** The ratio of correctly predicted positive observations to all actual positives. High recall is crucial when missing a positive is costly, such as in medical diagnosis.
- **F1 Score** ($2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$)**:** The harmonic mean of precision and recall, providing a single score that balances both, especially useful for imbalanced datasets. Labelf +6

When to Use Which Metric

- **Accuracy:** Use when classes are balanced, and false positives/negatives have similar costs.
- **Precision:** Use when minimizing false positives is critical (e.g., spam detection).
- **Recall:** Use when minimizing false negatives is critical (e.g., cancer detection).
- **F1 Score:** Use when you need a balance between precision and recall, or when the dataset is highly imbalanced. Labelf +4

A confusion matrix is a table, where is the number of target classes, used to evaluate the performance of a classification model by comparing predicted labels against true labels. It breaks down results into true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN),

Gradient Descent is an optimization algorithm used to **minimize a loss function** by iteratively moving parameters in the direction of **steepest decrease**.

Description [edit]

Gradient descent is based on the observation that if the multi-variable function $f(\mathbf{x})$ is defined and differentiable in a neighborhood of a point $\mathbf{a}$, then $f(\mathbf{x})$ decreases *fastest* if one goes from $\mathbf{a}$ in the direction of the negative gradient of f at $\mathbf{a}$, $-\nabla f(\mathbf{a})$. It follows that, if

$$\mathbf{a}_{n+1} = \mathbf{a}_n - \eta \nabla f(\mathbf{a}_n)$$

Gradient descent is a method for unconstrained mathematical optimization. It is a first-order iterative algorithm for minimizing a differentiable multivariate function.

Variants of Gradient Descent

Method	Idea
Batch Gradient Descent	Uses all training data
Stochastic Gradient Descent (SGD)	Uses one sample at a time
Mini-batch Gradient Descent	Uses small batches

Mini-batch is most common in deep learning.

Geometrically: **a bowl shape**.

Soft Requirement on Convexity

Gradient descent **does not require convexity**, but convexity gives **strong guarantees**.
Without convexity:

- Multiple minima may exist
- Algorithm might converge to **local minima**

However it **often still works well**.

When Convexity Does Not Matter Much

Convexity matters less in **large overparameterized models**.
Examples:

- Deep neural networks
- Large transformer models
- CNNs

Reasons:

1. High-dimensional spaces contain many **good minima**
2. Most local minima have similar loss
3. SGD noise helps escape poor minima
4. Saddle points are more common than bad minima

When Gradient Descent Fails
Problems occur when:

Problem	Explanation
Learning rate too large	Divergence
Learning rate too small	Very slow training
Non-smooth loss	Gradient unstable
Poor conditioning	Zig-zag updates

Modern Improvements
To solve gradient descent limitations, optimizers were developed:

Optimizer	Idea
Momentum	Smooth updates
RMSProp	Adaptive learning rates
Adam	Momentum + adaptive scaling

What is a convolution?
Convolution is a fundamental mathematical operation that combines two functions to produce a third, expressing how the shape of one is modified by the other.

Hypothesis generation.
Hypothesis generation is the crucial initial step in research and analysis, involving the formulation of testable, educated, and tentative answers to research questions

Error analysis.

Explain how the autocorrelation of a positive-trend line can be negative.

Autocorrelation measures the relationship (correlation) between a variable's current value and its past values (lagged versions) over time, indicating how similar a dataset is to a delayed copy of itself.
5. Geometric Interpretation
For a trend:

time →
1 2 3 4 5

Centered around the mean:

-2 -1 0 1 2

Large shifts align:

-2 with 2
-1 with 1

Opposite signs → **negative correlation**.

ANOVA, multivariate analysis, PCA / Factor Analysis, population tests and comparison.

Analysis of Variance (ANOVA) is a statistical method used to compare the means of three or more independent groups to determine if at least one is significantly different

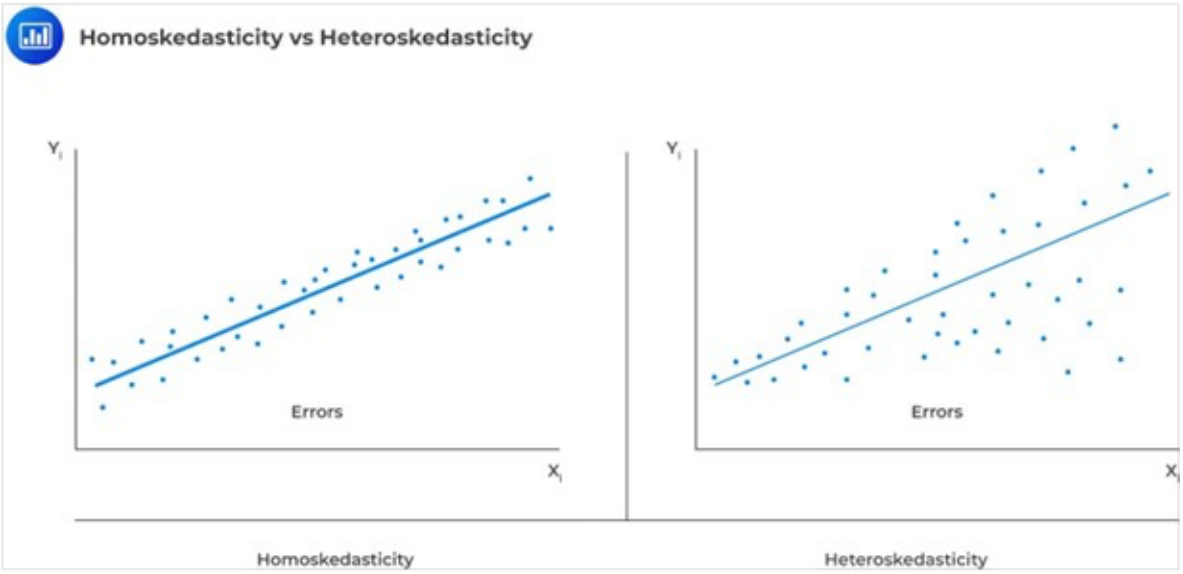
Factor analysis is a multivariate statistical method used to reduce a large number of observed, correlated variables into a smaller, more manageable set of unobserved "latent factors"
FA tries to reduce the number of variables while still being able to reproduce the original correlation matrix as best as possible.

Population tests compare sample data to a standard value (single sample) or compare two distinct populations to determine if differences are statistically significant. Common methods include t-tests for means (paired or independent), ANOVA for multiple groups, z-tests for proportions, and non-parametric tests like Wilcoxon/Mann-Whitney for skewed dat

2. What if Regression Has Heteroskedasticity?
Heteroskedasticity
The **variance of the error changes across observations**.

Robust standard errors	Correct inference
Transform variables	e.g., log transform
Model variance explicitly	GLS

Heteroskedasticity occurs in regression analysis when the variance of the error terms (residuals) is not constant across all levels of the independent variables. It violates the homoscedasticity assumption of Ordinary Least Squares (OLS) regression, causing standard errors to be inaccurate, which renders hypothesis tests and confidence intervals unreliable, though coefficient estimates remain unbiased



To fix heteroscedasticity, transform the dependent variable (e.g., using natural logs or square roots) t

How Do You Avoid Overfitting?
Overfitting
The model **memorizes training data** but performs poorly on new data.
Mathematically:
Training error ↓
Test error ↑

Main techniques

Method	Idea
Regularization	Penalize large weights
Cross-validation	Estimate generalization
Early stopping	Stop training before overfitting
More data	Reduces variance
Simpler models	Reduce complexity
Dropout (NN)	Randomly disable neurons

Method 1 — Logistic Regression

Transform regression output into probabilities:

$$P(y = 1|x) = \sigma(w^T x + b)$$

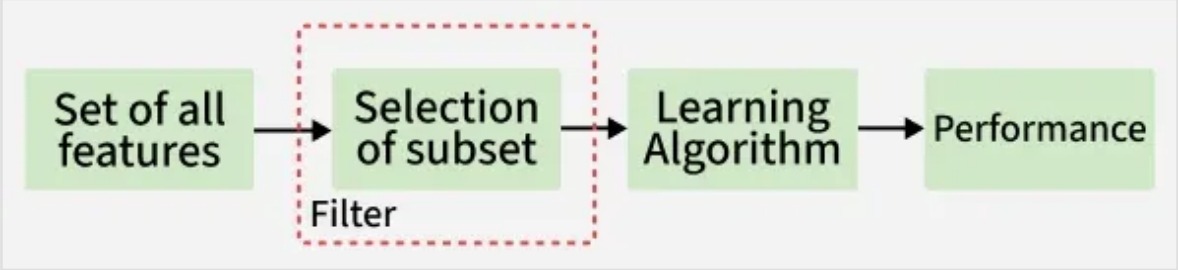
Then classify:

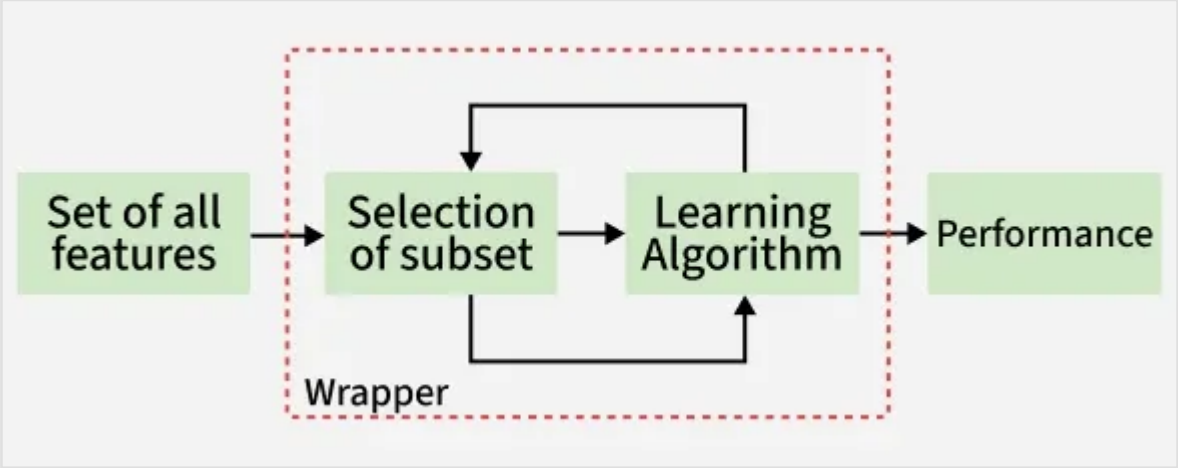
$$y = \begin{cases} 1 & \text{if } P > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Method 2 — Softmax (Multiclass)

For K classes:

$$P(y = k|x) = \frac{e^{w_k^T x}}{\sum_{j=1}^{K} e^{w_j^T x}}$$





- Handling "too many features" (Is that a problem? When? What would you do about it?)
How can you choose the value of k on k-means?

Transformer Basics: Understands the "Encoder-only" (BERT) vs. "Decoder-only" (GPT)
Understands Embeddings as an extension to "Feature Engineering."
LLM
Workflow vs. Agent: Understands the difference between a fixed Workflow (Step A -> Step
B) and a dynamic Agent (Goal -> Reason -> Action).

Workflow vs Agent

Aspect	Workflow	Agent
Definition	A **predefined sequence of steps** executed in a fixed order	An **autonomous system that decides actions dynamically**
Control	Deterministic	Adaptive / decision-based
Execution	Follows a fixed pipeline	Chooses next step based on state or goal
Flexibility	Low	High
Complexity	Easier to design and debug	More complex behavior
Use case	Data pipelines, ML training	Autonomous AI systems

-Vector DB & other types of Retrieval: identifying when to use each based on trade-offs in latency, scale, and metadata filtering requirements.

One-sentence intuition

- **Keyword retrieval** finds documents with the same words.
- **Vector retrieval** finds documents with the same **meaning**.

Encoding & Token Awareness: Understands how different models "count" tokens and how that impacts both cost and performance.
- Context Precision & Recall: Did we retrieve only relevant documents? Did the retrieval step actually include the document containing the answer?

Summary

Retrieval Type	Representation	Matching
TF-IDF	sparse	keyword
BM25	sparse	keyword ranking
Dense retrieval	embeddings	semantic similarity

Vector DB	embedding index	nearest neighbor
Hybrid retrieval	sparse + dense	combined score

✅ **One-sentence intuition**

- **Keyword retrieval** finds documents with the same words.
- **Vector retrieval** finds documents with the same **meaning**.

Documento
↓
Chunking (bloques de texto)
↓
Tokenization
↓
Embedding
↓
Vector DB

ONLINE STAGE
user query
↓
embedding
↓
vector search
↓
top-k documents
↓
(optional reranker)
↓
LLM prompt
↓
generated answer

Add context to the model using prompting techniques: Zero and few shots, prompt chaining, etc.

1. Zero-Shot Prompting

Idea

Ask the model to perform a task **without giving examples**, only instructions.

Structure

```
Instruction + Input
```

Example

```
Classify the sentiment of this sentence as positive or
```

```
negative.

Sentence: The movie was fantastic.
```

Output:

```
Positive
```

When it works well

- LLMs trained on large datasets
- simple tasks
- general reasoning

2. Few-Shot Prompting
Idea
Provide **a few examples** of input-output pairs so the model learns the pattern.
Structure

Example 1
Example 2
Example 3
New query

Example

Sentence: I love this product.
Sentiment: Positive

Sentence: This is terrible.
Sentiment: Negative

Sentence: The service was amazing.
Sentiment:

The model infers the pattern and answers.

Why it works
The model learns **the task format and reasoning pattern** from examples.

Chain-of-Thought Prompting (CoT)
Idea
Ask the model to **show intermediate reasoning steps**.
Example
Prompt:

```
Solve step by step:
```

```
If a train travels 60 km/h for 2 hours, how far does it go?
```

Output:

```
Distance = speed × time
Distance = 60 × 2
Distance = 120 km
```

Why it works:

- encourages structured reasoning
- improves performance on complex tasks

Software Development

- What is the difference between "checkout", "add", "commit" and "branch" in Git?
  -
  - What are the 4 main parts of an SQL statement?

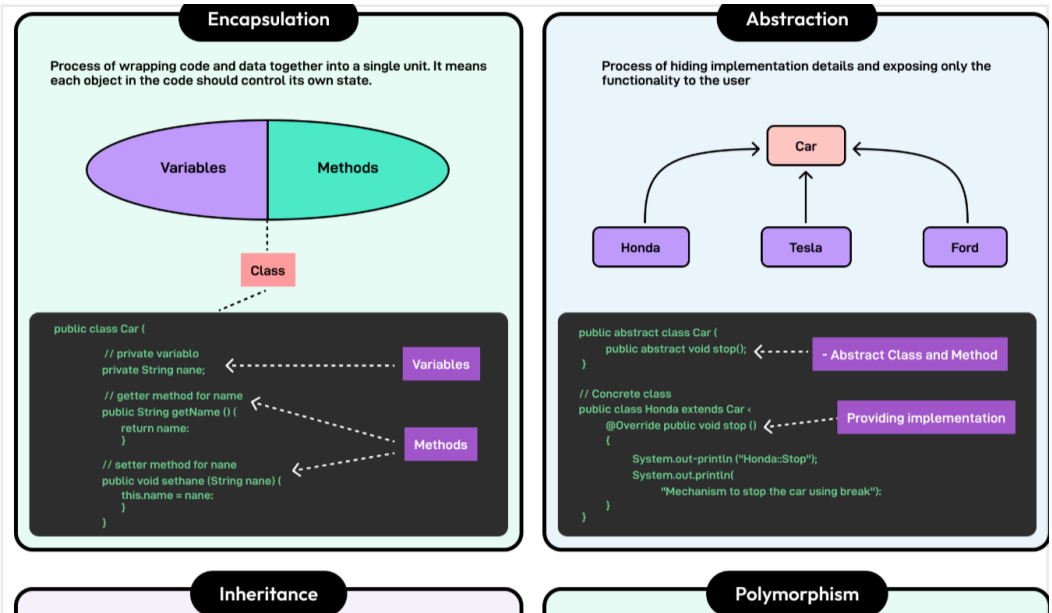
Python

- Basic API development: FastAPI, Flask, Eventlet / Gunicorn. REST, wsgi.

Visualizations: Bokeh, Plotly, others.: Bokeh y Plotly son dos de las librerías de visualización de datos más potentes y populares basadas en Python, diseñadas específicamente para crear gráficos interactivos, modernos y orientados a la web.

-
-
-

Fundamentals of Object Oriented Programming.
Packaging classes and methods into data science libraries.



- An abstract class is a restricted, partially implemented class in object-oriented programming that cannot be instantiated directly and is designed to be inherited. It serves as a blueprint for subclasses, allowing a mix of fully implemented concrete methods and abstract methods (lacking body) that subclasses must implement.

- **Key Aspects of Abstract Classes:**
- **No Instantiation:** You cannot create an instance (object) of an abstract class using new.
- **Abstract Methods:** They can contain methods without a body (abstract methods), which forcing subclasses to provide specific implementations.
- **Concrete Methods:** They can also contain fully implemented methods, fields, and constructors.
- **Inheritance:** Used to define a common, mandatory structure for derived classes (e.g., a Shape base class for Circle and Square).
–
- **Extend (Inheritance)**
- **Definition:** A mechanism where a new class (subclass) inherits fields and methods from an existing class (superclass).
- **Purpose:** To reuse code and create an "IS-A" relationship (e.g., a Dog extends Animal).
- **Capabilities:** The subclass can add new fields/methods and use inherited ones.
- **Keyword:** extends (Java), : (C#).
-
- **Override (Polymorphism)**
- **Definition:** A feature allowing a subclass to provide a specific implementation of a method already defined in its superclass.
- **Purpose:** To modify or specialize the behavior of an inherited method.
- **Requirements:** The method signature (name, parameters, return type) must be the same.
- **Polymorphism:** It enables runtime polymorphism, where the subclass method is called even if the reference is of the superclass type.
–
–

  Literate programming.
  Reproducible research.
  Error catching, context manager, generators and comprehensions, reusable methods and classes, some code optimization and profiling.
–
- **Using a context manager**
-
- with open("data.txt", "r") as f:
- data = f.read()

-
- What happens internally:
- File is opened
- Code block executes
- File automatically closes
- Equivalent behavior:
-
- f = open("data.txt", "r")
- try:
- data = f.read()
- finally:
- f.close()

```
import time

class Timer:

    def __enter__(self):
        self.start = time.time()
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        end = time.time()
        print("Elapsed:", end - self.start)

with Timer():
    time.sleep(2)
```

```
from contextlib import contextmanager.   #Using a decorator
import time

@contextmanager
def timer():
    start = time.time()
    yield
    end = time.time()
    print("Elapsed:", end - start)
```

Concept	Purpose
Generator	produce values lazily using yield
Generator expression	compact generator syntax
List comprehension	create lists concisely
Set comprehension	create sets

Dict comprehension	create dictionaries

. When Generators Are Useful
Generators are ideal for:

- streaming data
- large files
- pipelines
- infinite sequences

1. Generators
Meaning
A **generator** is a function that **produces values one at a time instead of computing them all at once**.
It uses the keyword **yield**.
This makes generators **memory efficient**, because values are generated **lazily** (only when needed).

Time Profiling with cProfile

Example:

```
import cProfile

def slow_function():
    total = 0
    for i in range(1000000):
        total += i
    return total

cProfile.run("slow_function()")
```

Output shows:

- number of calls
- total time
- cumulative time per function

Example result (simplified):

Function	Calls	Time
slow_function	1	0.12 s

Memory Profiling
Library: memory_profiler
Example:

```
from memory_profiler import profile

@profile
def allocate():
   a = [i for i in range(1000000)]
   return a
```

Shows **memory usage per line**.

Category	Technique	What it does	Example	Benefit
Profiling	cProfile	Measures execution time of functions	cProfile.run("func()")	Finds slow functions
Profiling	line_profiler	Measures execution time per line	@profile decorator	Detects exact bottleneck lines
Profiling	memory_profiler	Tracks memory usage	@profile on functions	Identifies memory leaks
Benchmarking	timeit	Tests performance of small code snippets	timeit.timeit(stmt, number=1000)	Accurate microbenchmarks
Loop Optimization	List comprehensions	Replace loops with compact expressions	[x*x for x in range(10)]	Faster and cleaner
Built-ins	Use built-in functions	Replace manual loops	sum(data) instead of loop	Faster (C-optimized)
Data Structures	Choose efficient structures	Use sets/dicts for lookup	x in my_set	(O(1)) lookup vs (O(n))
Memory	Generators	Produce values lazily	(x*x for x in range(n))	Lower memory usage
Vectorization	Use NumPy operations	Replace Python loops with vector ops	arr * 2	Huge speed improvement

Caching	Memoization	Store previous results	@lru_cache()	Avoid recomputation
Algorithms	Better algorithm complexity	Reduce computational complexity	binary search	Major speed improvements
Parallelism	Multiprocessing	Run tasks on multiple CPUs	Pool.map()	Faster CPU tasks
Concurrency	Async / threading	Handle I/O efficiently	asyncio	Faster I/O workflows

Basic Python Engineering: Able to write clean functions that can be used as "Tools" by an AI.

- Basic Agent Tool: LangChain, Smolagents, etc

Principle	Description	Example
Single responsibility	Function should do **one clear task**	get_weather(city)
Clear inputs	Use explicit parameters	def convert_temp(celsius: float)
Structured output	Return predictable format (dict/JSON)	{"temperature": 20}
Deterministic behavior	Same input → same output	Avoid randomness unless needed
Error handling	Validate inputs	if city is None: raise ValueError
Type hints	Improve readability and tooling	city: str -> dict
Docstrings	Explain what the tool does	describe inputs and outputs
No side effects	Avoid modifying global state	pure functions preferred
Small functions	Easier for AI systems to compose	keep logic short

LLM and Generative AI

-Knows and use of generative AI-based tools.

Basic prompting techniques. Prompt Hygiene.

The 7-Point Prompt Hygiene Checklist (What to Avoid)

Before submitting any prompt, check for these seven risks:

1. **Full Names:** Yours or others'.
2. **Private Contact Info:** Addresses, phone numbers, or emails.
3. **Security Data:** Passwords, API keys, or security protocols.
4. **Financial/Legal Data:** Sensitive, proprietary company or personal financial information.
5. **Medical Details:** PHI (Protected Health Information).
6. **Confidential Work Data:** Internal, unreleased documents or code.
7. **Clipboard Content:** Accidental pastes of private information.